



Object Oriented Concepts using C++

Trainer : Priyanka R. Rangole

Email: priyanka.rangole@sunbeaminfo.com



Main Function

- main should be entry point function of C/C++
- Calling/invoking the main function is the responsibility of the operating system.
- Hence it is also called as Callback function.

[In cpp extension of file is -> .cpp
compiler name is -> g++]

Helloworld.cpp

Functions / User Defined Functions

- It is a set of instructions written to gather as a block to complete specific functionality.
- Function can be reused.
- It is a subprogram written to reduce complexity of source code
- Function may or may not return value.
- Function may or may not take argument
- Function can return only one value at time
- **Writing function helps to**
 - improve readability of source code
 - helps to reuse code
 - reduces complexity
- **Types of Functions**
 - Library Functions
 - User Defined Functions

User Defined Functions

A function is a group of statements that together perform a task.

- **Function declaration / Prototype / Function Signature**

<return type> <functionName> ([<arg type>...]);

- **Function Call**

<location> = < functionName >(<arg value/address>);

- **Function Definition**

<return type> < functionName > ([<arg type> <identifier>...])

{

 //function body

}

Inline Function

- C++ provides a keyword *inline* that makes the function as an inline function.
- Inline functions get replaced by the compiler at its call statement. It ensures faster execution of function.
- Inline is a request made to the compiler.
- If a function is inline, the compiler places a copy of the code of that function at each point where the function is called at compile time.

When to use Inline function?

- We can use the inline function when performance is needed.

Default Arguments

- In C++, functions may have arguments with the default values. Passing these arguments while calling a function is optional.
- A default argument is a default value provided for a function parameter/argument.
- If the user does not supply an explicit argument for a parameter with a default argument, the default value will be used.
- If such an argument is not passed, then its default value is considered. Otherwise arguments are treated as normal arguments.
- Default arguments should be given in right to left order.

```
Int sum (int a, int b, int c=0, int d=0) { return  
    a + b + c + d;  
}
```

The above function may be called as

- Res=sum(10,20);
- Res=sum(10,20,40);
- Res=sum(10,30,40,50);

Function Overloading

- Functions with the same name and different signature are called overloaded functions.
- Return type is not considered for function overloading.
- Function calls are resolved according to types of arguments passed.
- Function overloading is possible due to name mangling done by the C++ compiler (Name mangling process , mangled name)
- Differ in number of input arguments
- Differ in data type of input arguments
- Differ at least in the sequence of the input arguments

Example :

```
void print ( int n1) {      }
void print (char a) {      }
void print (int a, int b) {      }
void print (char a, int b) {      }
void print (int i, char c) {      }
```

class

Time class in CPP

Class is collection of logically related data members and member functions.

Difference between class and struct in cpp

- all the members of class are by default private and all the members of structure in cpp are by default public.

Class

- Building block that binds together data & code.
- Program is divided into different classes
- Class is a collection of data members and member functions.
- Class represents a set/group of such objects which has common structure and common behaviour.
- Class is a logical entity.
- Class has
 - Variables (data members)
 - Functions (member functions or methods)
- By default class members are private(not accessible outside class scope)
- Classes are stand-alone components & can be distributed in form of libraries
- Class is blue-print of an object.

Struct in CPP

```
struct Time
{
    int hr;
    int min;
    int sec;
    void printTime() {--}
    void acceptTime() {--}
    void incrementTimeByOneSec() {--}
}
```

Data member and Member function

Data member - also called field,
property, attribute

Member function-methods , operations,
behaviour, message

```
struct Time
```

```
{
```

```
    int hr;
```

```
    int min;
```

```
    int sec;
```

```
    void printTime() {--} void
```

```
    acceptTime() {--} void
```

```
}    incrTime() {--}
```

Access Specifier

- - If we want to control visibility of members of a structure/class then we should use access Specifier.
- - Defines the accessibility of data member and member functions
- **Access specifiers in C++**
 1. private(-)
 2. protected(#)
 3. public(+)
- 1. Private - Can access inside the same struct/class in which it is declared Generally data members should be declared as private. (data security)
- 2. public - Can access inside the same struct/class in which it is declared as well as inside out side function(like main()). Generally member functions should be declared as public.

Access Specifier

-

Access Specifier	Same Class	Derived Class	Non Member Function
private	A	NA	NA
protected	A	A	NA
public	A	A	A

- In C++, structure members are by default public and class members are by default private

Difference - Structure in C & C++

struct in c	struct in c ++
we can include only variables into the structure	we can include the variables as well as the functions in structure.
We need to pass a structure variable by value or by address to the functions	We don't pass the structure variable to the functions to accept it / display it. The functions inside the struct are called with the variable and DOT operator
By default all the variables of structure are accessible outside the structure. (using structure variable name)	By default all the members are accessible outside the structure, but we can restrict their access by applying the keywords private /public/ protected.
structTime t1	structTime t1;
AcceptTime(struct Time &t1);	t1.AcceptTime(); //function call

class

Time class in CPP

Class is collection of logically related data members and member functions.

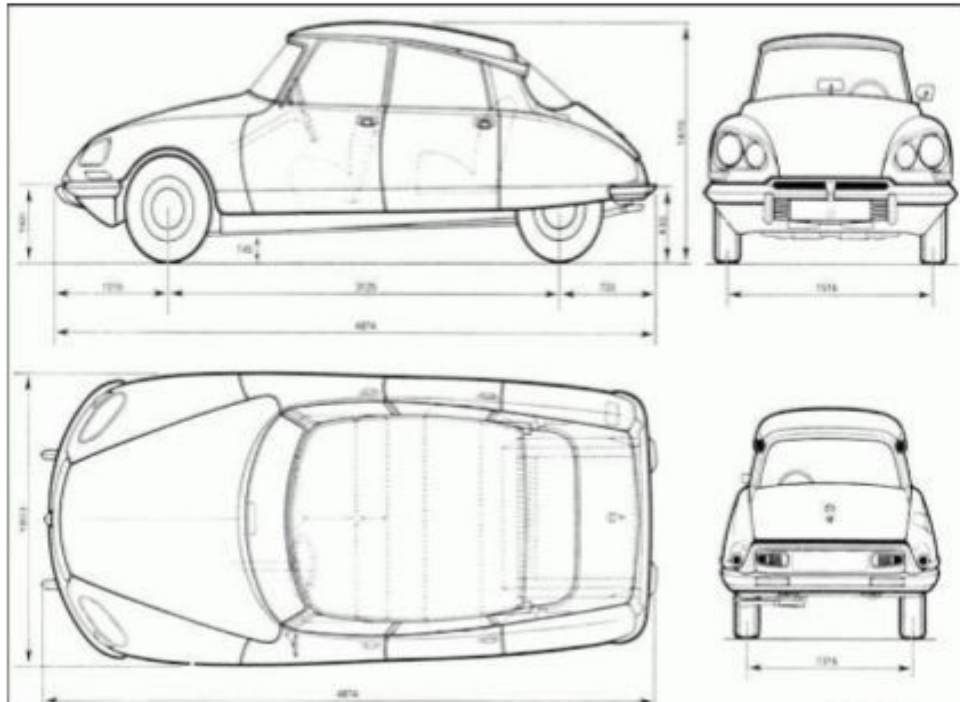
Difference between class and struct in cpp

- all the members of class are by default private and all the members of structure in cpp are by default public.

Class

- Building block that binds together data & code.
- Program is divided into different classes
- Class is a collection of data members and member functions.
- Class represents a set/group of such objects which has common structure and common behaviour.
- Class is a logical entity.
- Class has
 - Variables (data members)
 - Functions (member functions or methods)
- By default class members are private(not accessible outside class scope)
- Classes are stand-alone components & can be distributed in form of libraries
- Class is blue-print of an object.

Blue Print of Car



Objects of a
Class Car



Blue Print of building



Real life entity(Object)



Object

- An object is an instance of class.
- An entity that has physical existence, can store data.
- An entity, which gets space inside memory is called an object.
- Object is used to access data members and member function of the class
- The process of creating objects from a class is called instantiation.

SunBeam

Object

- **Object has**

- ***State* of object (Data members)**

- The value stored inside an object is called the state of the object.
 - The value of the data member represents the state of the object.

- ***Behavior* of object (Member function)**

- The set of operations that we perform on an object is called behaviour of an object.
 - Member functions of class represent behaviour of the object.

- ***Identity* of object (Unique address)**

- The value of any data member, which is used to identify an object uniquely is called its identity.
 - The state of the object may be the same but its address can be **Unique** considered as its identity.

Few Points to note

- Member functions do not get space inside objects.
- If we create an object of the class then only data members get space inside the object. Hence size of object depends on size of all the data members declared inside class.
- Data members get space once per object according to the order of data member declaration.
- The structure of the object depends on data members declared inside class.
- Member functions do not get space per object rather it gets space on code segments and all the objects of the same class share a single copy of it.
- Member function's of the class defines behaviour of the object

Class

Empty class

A class which does not have any Data member or member function is called the Empty class.

- We can create an object of an empty class.
- The size of an object of an empty class is 1 byte.

this pointer

Structure in C

```
struct time {  
    int hr, min, sec;  
};  
  
void accept( struct time *p) {  
    scanf("%d:%d:%d",  
        &p->hr, &p->min, &p->sec);  
}  
Main()  
{  
    struct time t;  
    accept(&t);  
}
```

class in C++

```
class time {  
    int hr, min, sec;  
  
    void accept(){  
        scanf("%d:%d:%d",&hr, &min, &sec);  
    }  
};  
  
Main()  
{  
    time t;  
    t.accept();  
}
```

this pointer

- To process the state of the object we should call the member function on the object. Hence we must define member functions inside class.
- If we call a member function on an object then the compiler implicitly passes the address of that object as an argument to the function implicitly.
- To store the address of an object, the compiler implicitly declares one pointer as a parameter inside member function. Such a parameter is called this pointer.
- `this` is a keyword. "`this`" pointer is a constant pointer.
- `this` is used to store the address of the current object or calling object.
- The invoking object is passed as an implicit argument to the function.
- *this* pointer points to the current object i.e. object invoking the member function.
- Thus every member function receives *this* pointer.
- Following functions do not get this pointer:
 1. Global Function
 2. Static Member function
 3. Friend Function.

Types of Member Functions within class

- Constructor : object initialization
- Destructor : used to release the resources
- Mutators / setter : modify state of object
- inspector/getter : read the data member but do not change the state of the object
- Facilitator : Provide extra facility to work with object

Constructor

- It is a member function of a class which is used to initialize objects.
- The constructor has the same name as that of class and doesn't have any return type.
- Constructors get automatically called when an object is created i.e. memory is allocated to the object.
- If we don't write any constructor, the compiler provides a default constructor.
- Due to following reasons, constructor is considered as special function of the class:
 1. Its name is the same as the class name.
 2. It doesn't have any return type.
 3. It is designed to call implicitly.
 4. In the lifetime of the object , it gets called only once per object and according to the order of its declaration.

Constructor

- We can not call constructors on objects, pointers or references explicitly. It is designed to call implicitly.
- We can not declare constructors static, constant, volatile or virtual. We can declare constructors only inline.
- Constructor overloading means inside a class more than one constructor is defined.
- We can have constructors with
 - No argument : initialize data member to default values
 - One or more arguments : initialize data member to values passed to it
 - Argument of type of object : initialize object by using the values of the data members of the passed object. It is called as copy constructor.

Types of Constructor

- Parameterless constructor
 - also called zero argument constructor or user defined default constructor
 - If we create object without passing argument then parameterless constructor gets called
 - Constructor do not take any parameter
- Parameterized constructor
 - If constructor take parameter then it is called parameterized constructor
 - If we create object, by passing argument then parameterized constructor gets called
- Default constructor
 - If we do not define a constructor inside the class then the compiler generates a default constructor for the class.
 - The default constructor is parameterless.

Thank You

Sunbeam

